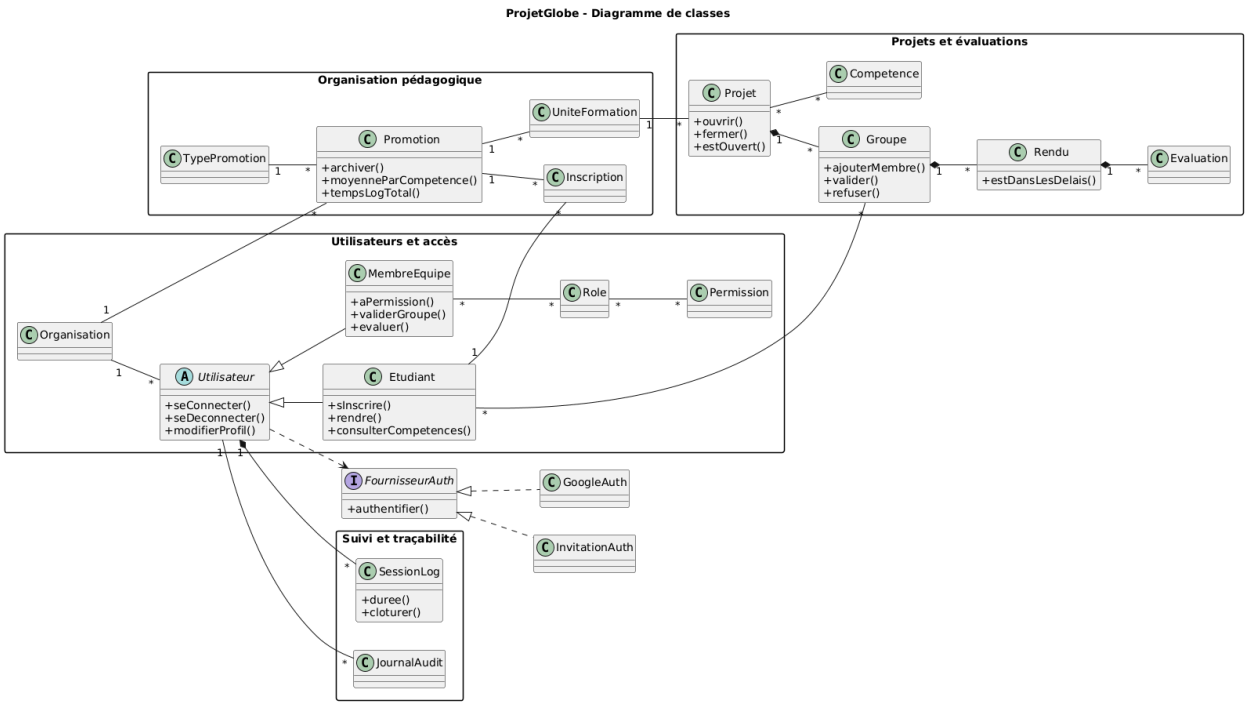
Diagramme de classes - ProjetGlobe

Le diagramme de classes décrit la structure de l'application : 16 classes métier, qui correspondent une à une aux 16 entités du MCD, plus les classes techniques d'authentification et les énumérations. Il appartient, avec le schéma de base de données, à la documentation **structurelle** ; les diagrammes de la section 4 en sont la partie **comportementale**.

1. Diagramme de classes



Les classes sont regroupées en 4 paquetages qui correspondent aux 4 groupes de tables du MPD. Les multiplicités UML se lisent à l'inverse des cardinalités Merise : celle placée près d'une classe indique combien d'objets de cette classe sont liés à un objet de l'autre. Exemple : `Projet "0..*" -- "1..*" Competence` signifie qu'un projet vise au moins une compétence, ce qui correspond à VISER (1,N) côté PROJET dans le MCD.

2. Cohérence avec la base de données

Chaque entité du MCD se retrouve dans une classe, avec les mêmes attributs (en camelCase) et des multiplicités qui reprennent les cardinalités. Les associations porteuses de données deviennent une classe d'association ; les associations simples deviennent des liens.

Entité ou association (MCD)	Classe (UML)	Méthodes clés	Exigences
ORGANISATION	`Organisation`	—	ENF-08
UTILISATEUR (parent XT)	`Utilisateur` (abstraite)	`seConnecter()`, `seDeconnecter()`, `modifierProfil()`	EX-01, EX-02, EX-05
ETUDIANT	`Etudiant` hérite de `Utilisateur`	`sInscrire()`, `rendre()`, `consulterCompetences()`, `exporterMesDonnees()`	EX-14, EX-19, EX-24, RGPD
MEMBRE_EQUIPE	`MembreEquipe` hérite de `Utilisateur`	`aPermission()`, `validerGroupe()`, `evaluer()`	EX-04, EX-17, EX-22
ROLE, PERMISSION	`Role`, `Permission`	`accorder()`	EX-03, EX-04
TYPE_PROMOTION	`TypePromotion` + énum. `Modalite`	—	EX-07
PROMOTION	`Promotion`	`archiver()`, `moyenneParCompetence()`, `tempsLogTotal()`	EX-06, EX-08, EX-27
INSCRIRE (porte des données)	`Inscription` (classe d'association) + énum. `StatutParcours`	—	EX-09 (redoublement)
UNITE_FORMATION	`UniteFormation`	—	EX-11
PROJET	`Projet` + énum. `StatutProjet`	`ouvrir()`, `fermer()`, `estOuvert()`	EX-12, EX-13
COMPETENCE	`Competence`	—	EX-22, EX-24
GROUPE	`Groupe` + énum. `StatutGroupe`	`ajouterMembre()`, `valider()`, `refuser()`	EX-14 à EX-18
RENDU	`Rendu`	`estDansLesDelais()`	EX-19, EX-20
EVALUATION	`Evaluation`	—	EX-22, EX-23
SESSION_LOG	`SessionLog`	`duree()`, `cloturer()`	EX-25, EX-26
JOURNAL_AUDIT	`JournalAudit`	—	ENF-04

Classes sans table : `FournisseurAuth`, `GoogleAuth`, `InvitationAuth` (comportement d'authentification, rien à stocker) et les 4 énumérations (valeurs contrôlées en base par des contraintes `CHECK`).

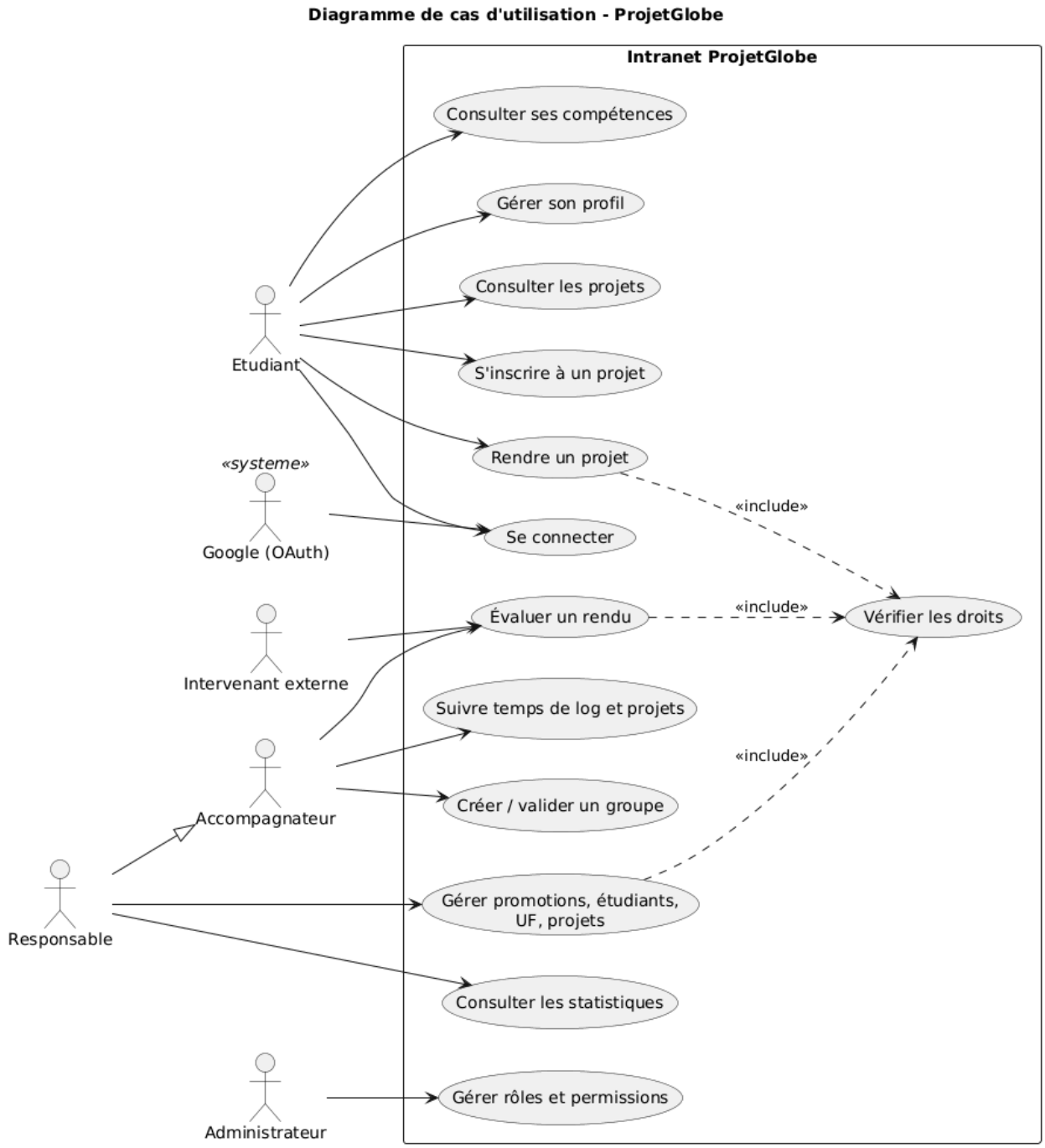
3. Choix de conception

Relation UML	Où	Pourquoi
Héritage	`Utilisateur` ← `Etudiant`, `MembreEquipe`	Connexion et profil communs, comportements propres à chaque spécialisation ; `Utilisateur` est abstraite car personne n'est « seulement » utilisateur (totalité du XT)
Réalisation (interface)	`FournisseurAuth` ← `GoogleAuth`, `InvitationAuth`	Le code d'authentification ne dépend que du contrat `authentifier()` ; ajouter un fournisseur (SSO d'un client SaaS) = ajouter une classe, sans modifier les autres
Composition	`Projet` ◆ `Groupe` ◆ `Rendu` ◆ `Evaluation`, `Utilisateur` ◆ `SessionLog`	Le cycle de vie de la partie dépend du tout : supprimer un projet supprime ses groupes, rendus et évaluations (`ON DELETE CASCADE` dans le MPD)
Classe d'association	`Inscription` entre `Etudiant` et `Promotion`	L'association porte des données (dates, statut de parcours) : c'est elle qui gère le redoublement
Association simple	`Organisation`, `UniteFormation`, `Role`, `Competence`…	Pas de dépendance de cycle de vie : on archive au lieu de supprimer
Dépendance	`Utilisateur` ..> `FournisseurAuth`	`seConnecter()` utilise un fournisseur sans le stocker

Règle de portée des droits : `MembreEquipe.aPermission(code)` vérifie l'union des permissions de ses rôles ; l'API vérifie en plus que la ressource fait partie de sa portée (promotion accompagnée, projet assigné). C'est ce double contrôle qui fait échouer, par exemple, un accompagnateur qui tente de supprimer une promotion.

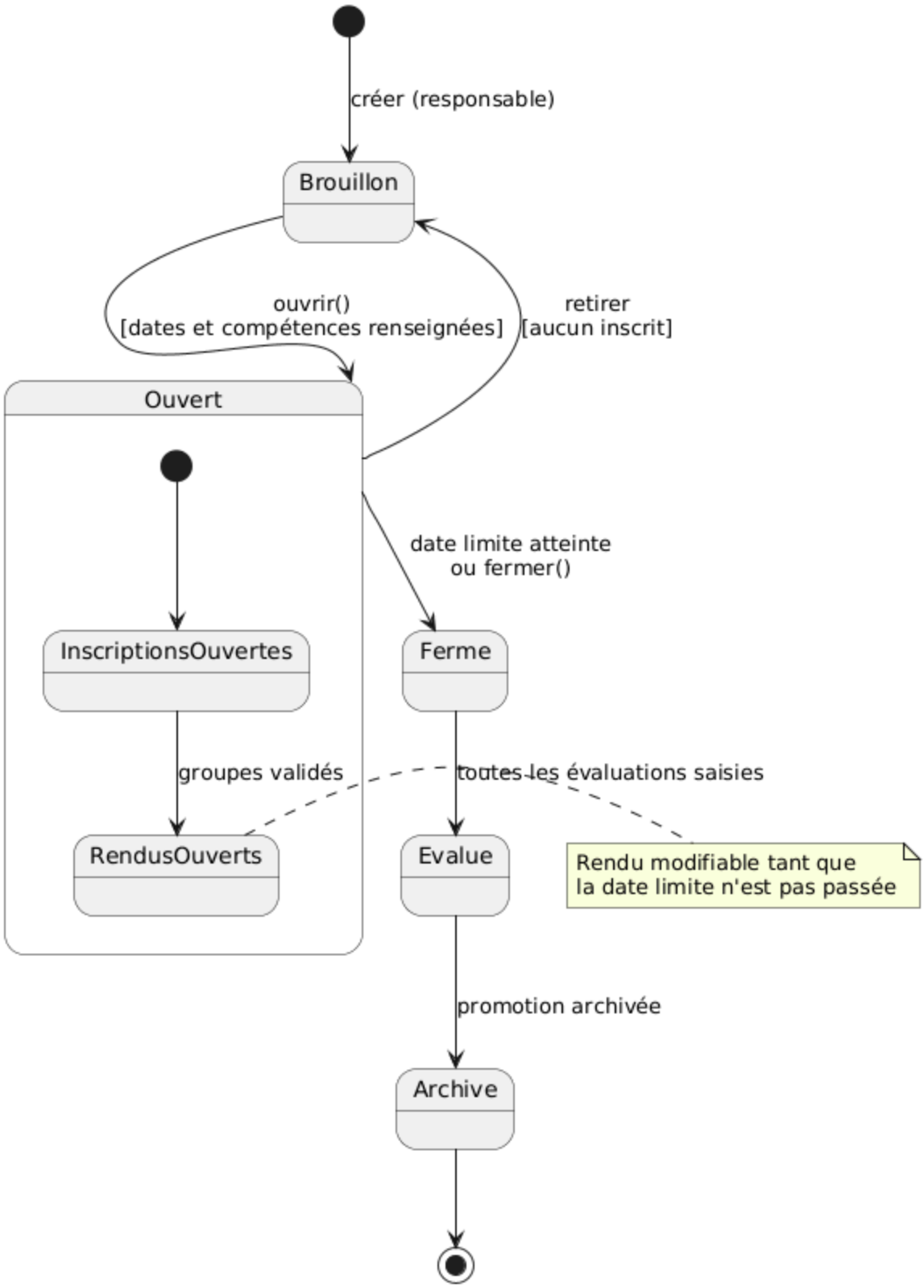
4. Documentation comportementale

Non exigée dans les rendus, elle décrit comment le système réagit. Trois diagrammes couvrent les points où les règles métier sont les plus fortes.



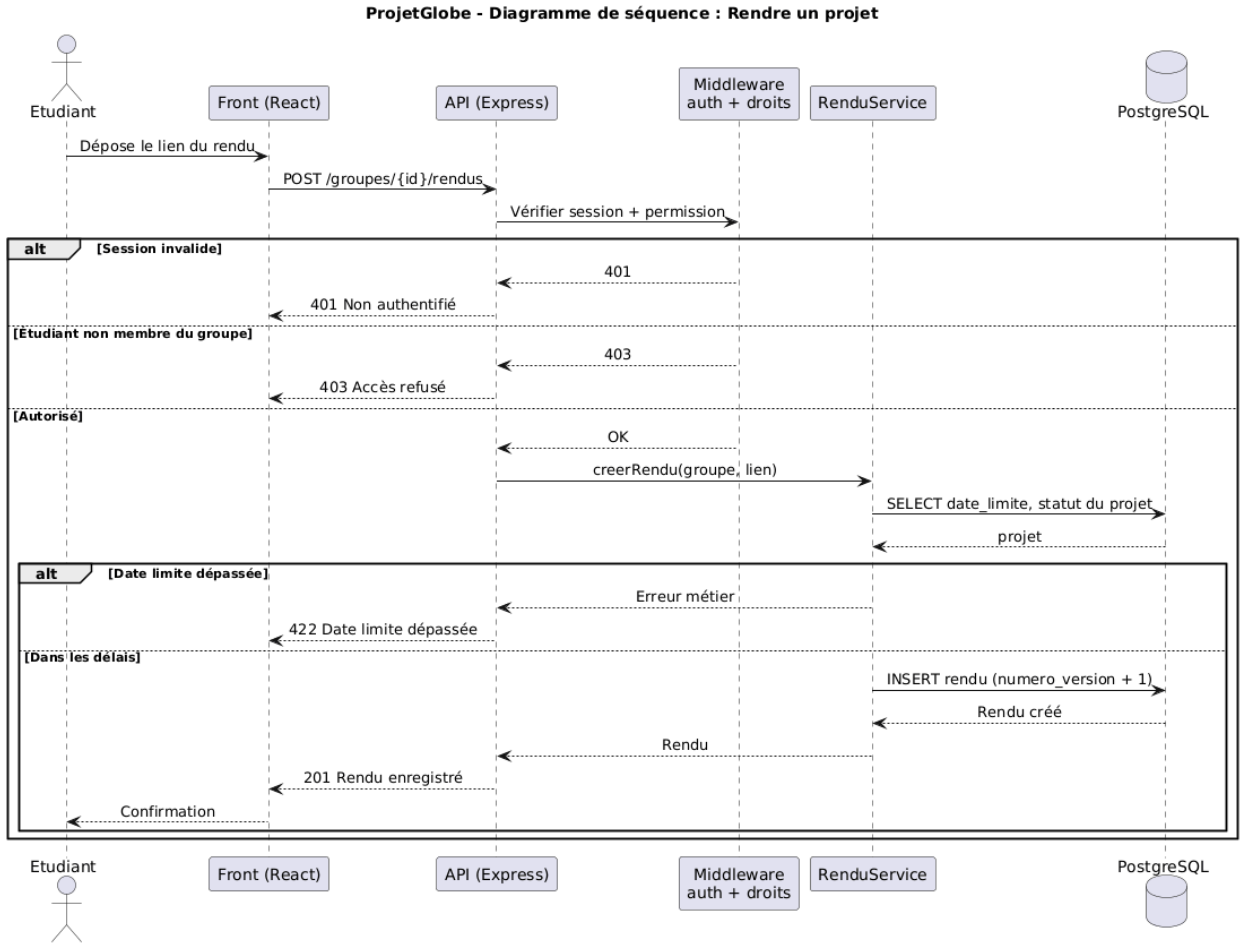
Le responsable hérite des cas de l'accompagnateur ; rendre, évaluer et gérer incluent tous la vérification des droits.

ProjetGlobe - Diagramme d'états d'un projet



Les états correspondent à la contrainte `projet.statut` du MPD et à l'énumération `StatutProjet`. Un rendu n'est possible que dans l'état « Ouvert », avant la date limite (EX-19).

Séquence : rendre un projet



Le diagramme montre les trois refus possibles, chacun testé en recette : session invalide (401), étudiant hors du groupe (403, REC-19c), date limite dépassée (422, REC-19b).

5. Scripts PlantUML

Les diagrammes sont écrits en PlantUML et versionnés dans `/modeles` : on modifie le texte, on régénère l'image. Ils évoluent avec le code (voir le plan de documentation just-in-time du CR de méthode).

Fichier	Contenu	Où le coller
`diagramme_classes.puml`	Diagramme de classes	plantuml.com, planttext.com, draw.io (Organiser > Insérer >

Fichier	Contenu	Où le coller
		Avancé > PlantUML) ou Lucidchart (import PlantUML)
`diagrammes_comportementaux.puml`	Cas d'utilisation, états d'un projet, séquence du rendu	Mêmes outils, un bloc `@startuml` … `@enduml` à la fois